

WEEK 7

Aim:

To develop a **Student Course Management System** using Node.js and Express.js that manages student information through HTTP requests, Express routes, URL parameters, query parameters, and JSON file operations.

Description:

This experiment demonstrates the development of a basic server-side web application using **Node.js and Express.js**. The application allows users to retrieve and search student information using different HTTP GET requests and Express routes. Node.js concepts such as **NPM, global objects, built-in modules, and file-system operations** are used along with Express.js concepts such as **environment setup, routing, HTTP methods, URL parameters, and query parameters**. Student data is stored in a JSON file and accessed using Node.js file-system operations.

Program:

Student Course Management System Using Node.js and Express.js

Program Code :

```
const express=require("express");
const app=express();
const fs=require("fs");
const PORT=3000;
const path=require('path');
app.get('/students',(req,res)=>{
let studentFilePath=path.join(__dirname,"students.json");

fs.readFile(studentFilePath,"utf8",(err,data)=>{
if(err){
```

```
console.log(err);
```

```
return;
```

```
}
```

```
let students=JSON.parse(data);
```

```
res.send(students);
```

```
});
```

```
});
```

```
app.get('/students/:id',(req,res)=>{
```

```
let studentFilePath=path.join(__dirname,"students.json");
```

```
fs.readFile(studentFilePath,"utf8",(err,data)=>{
```

```
if(err){
```

```
console.log(err);
```

```
return;
```

```
}
```

```
let id=req.params.id;
```

```
let students=JSON.parse(data);
```

```
let student="";
```

```
for(let i=0;i<students.length;i++){
```

```
if(students[i].id===id){
```

```
    student=students[i];
```

```
    break;
```

```
  }
```

```
}
```

```
if(!student){
```

```
  res.send("No student exists with ID :",id);
```

```
}
```

```
else{
```

```
  res.send(`Student name : ${student.name} , Student course =  
  ${student.course}`);
```

```
}
```

```
});
```

```
});
```

```
app.get('/search',(req,res)=>{
```

```
  let studentFilePath=path.join(__dirname,"students.json");
```

```
  fs.readFile(studentFilePath,"utf8",(err,data)=>{
```

```
    if(err){
```

```
      console.log(err);
```

```
      return;
```

```
    }
```

```
    let course=req.query.course;
```

```

let students=JSON.parse(data);
let result=[];

for(let i=0;i<students.length;i++){
    if(students[i].course.toLowerCase()==course.toLowerCase()){
        result.push(students[i]);
    }
}
if(result.length==0){
    res.send("No students are enrolled in the Course : ",course);
}
else{
    res.send(result);
}

});

});
app.get("/system",(req,res)=>{
    const os=require('os');
    let version=os.version();
    let platform=os.platform();
    let arc=os.arch();
    res.send(`Platform : ${platform} , Version : ${version} , Architecture : ${arc}`);

});
app.get("/dns",(req,res)=>{

```

```
const dns=require("dns");
dns.lookup("google.com",(err,address)=>{
  if(err){
    res.send(err);
    return ;
  }
  res.send(`Domain : google.com , IP Address : ${address}`);

});

});
app.listen(PORT,()=>{
  console.log("Server is listening on port ",PORT);
});
```

Data : Students.json

```
[
  {
    "id": 101,
    "name": "Lasya",
    "course": "Node.js"
  },
  {
    "id": 102,
    "name": "Rahul",
    "course": "Java"  },
]
```

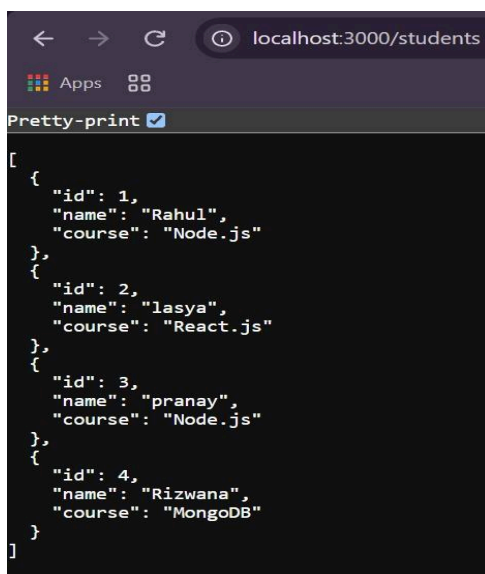
```
{
  "id": 103,
  "name": "Ani",
  "course": "MongoDB"
}
```

]

Execution Of the program

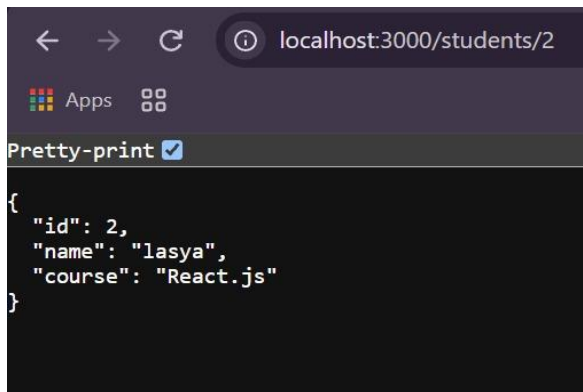
Method	URL/Route	Purpose
GET	http://localhost:3000/students	Retrieves all student records
GET	http://localhost:3000/students/:id	Retrieves a student using their ID
GET	http://localhost:3000/search?course=Node.js	Searches students based on course
GET	http://localhost:3000/system	Displays system/OS information
GET	http://localhost:3000/dns	Performs DNS lookup and displays IP information

Output: 1) Get all students: localhost:3000/students



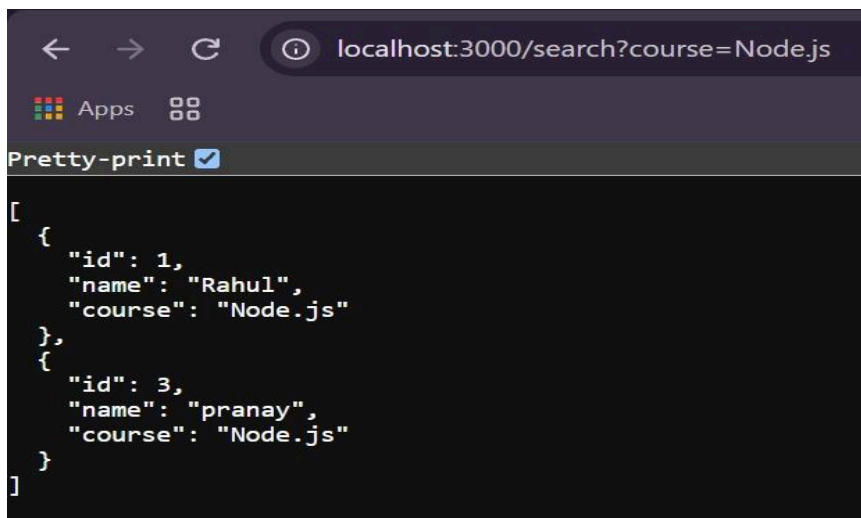
```
[
  {
    "id": 1,
    "name": "Rahul",
    "course": "Node.js"
  },
  {
    "id": 2,
    "name": "Iasya",
    "course": "React.js"
  },
  {
    "id": 3,
    "name": "pranay",
    "course": "Node.js"
  },
  {
    "id": 4,
    "name": "Rizwana",
    "course": "MongoDB"
  }
]
```

2) Get student with id =2 : localhost:3000/students/2



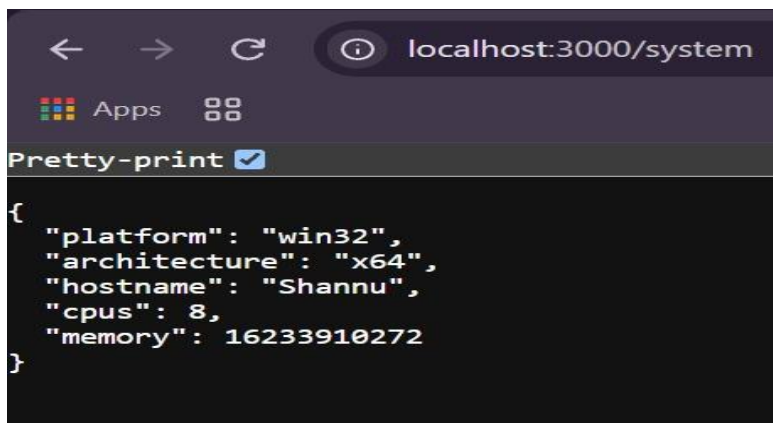
```
{
  "id": 2,
  "name": "lasya",
  "course": "React.js"
}
```

3) Get students with course **Node.js**: localhost:3000/search?course=Node.js



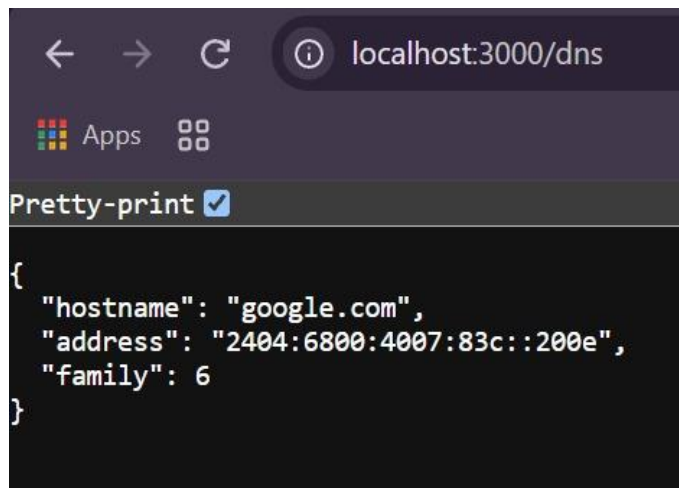
```
[
  {
    "id": 1,
    "name": "Rahul",
    "course": "Node.js"
  },
  {
    "id": 3,
    "name": "pranay",
    "course": "Node.js"
  }
]
```

4) Get system information : localhost:3000/system



```
{
  "platform": "win32",
  "architecture": "x64",
  "hostname": "Shannu",
  "cpus": 8,
  "memory": 16233910272
}
```

5) Get dns information : localhost:3000/dns



Result:

The Student Course Management System was successfully developed using Node.js and Express.js. The application demonstrates routing, HTTP GET methods, URL parameters, query parameters, built-in modules, file-system operations, and JSON-based student data management.